A complete trajectory $T_k(y_k, u_k)$ of the form (2.42) is generally feasible for only the subset $\hat{U}_k(y_k)$ of controls u_k that maintain feasibility:

$$\hat{U}_k(y_k) = \big\{u_k \mid T_k(y_k, u_k) \in C\big\}. \tag{2.43}$$

Our rollout algorithm starts from a given initial state $\tilde{y}_0 = \tilde{x}_0$, and generates successive partial trajectories

$$(\tilde{y}_1, \ldots, \tilde{y}_N),$$

of the form

$$\tilde{y}_{k+1} = \big(\tilde{y}_k, \tilde{u}_k, f_k(\tilde{x}_k, \tilde{u}_k)\big), \qquad k = 0, \ldots, N-1, \tag{2.44}$$

where $\tilde{x}_k$ is the last state component of $\tilde{y}_k$, and $\tilde{u}_k$ is a control that minimizes the heuristic cost $G\big(T_k(\tilde{y}_k, u_k)\big)$ over all u_k for which $T_k(\tilde{y}_k, u_k)$ is feasible. Thus at stage k, the algorithm forms the set $U_k(\tilde{y}_k)$ [cf. Eq. (2.43)] and selects from $U_k(\tilde{y}_k)$ a control $\tilde{u}_k$ that minimizes the cost of the complete trajectory $T_k(\tilde{y}_k, u_k)$:

$$\tilde{u}_k \in \arg \min_{u_k \in U_k(\tilde{y}_k)} G\big(T_k(\tilde{y}_k, u_k)\big); \tag{2.45}$$

see Fig. 2.5.2. The objective is to produce a feasible final complete trajectory $\tilde{y}_N$, which has a cost $G(\tilde{y}_N)$ that is no larger than the cost of $R(\tilde{y}_0)$ produced by the base heuristic starting from $\tilde{y}_0$, i.e.,

$$G(\tilde{y}_N) \le G\big(R(\tilde{y}_0)\big).$$

Note that $T_k(\tilde{y}_k, u_k)$ is not guaranteed to be feasible for any given u_k (i.e., may not belong to C), but we will assume that the constraint set $U_k(\tilde{y}_k)$ of problem (2.45) is nonempty, so that our rollout algorithm is well-defined. We will later modify our algorithm so that it is well-defined under the weaker assumption that just *the complete trajectory generated by the base heuristic starting from the initial state $\tilde{y}_0$ is feasible*, i.e., $R(\tilde{y}_0) \in C$.

Constrained Rollout Algorithm

The algorithm starts at stage 0 and sequentially proceeds to the last stage. At the typical stage k, it has constructed a partial trajectory

$$\tilde{y}_k = (\tilde{x}_0, \tilde{u}_0, \tilde{x}_1, \ldots, \tilde{u}_{k-1}, \tilde{x}_k) \tag{2.46}$$

that starts at the given initial state $\tilde{y}_0 = \tilde{x}_0$, and is such that

$$\tilde{x}_{t+1} = f_t(\tilde{x}_t, \tilde{u}_t), \qquad t = 0, 1, \ldots, k-1.$$

The algorithm then forms the set of controls

$$U_k(\tilde{y}_k) = \big\{u_k \mid T_k(\tilde{y}_k, u_k) \in C\big\}$$

that is consistent with feasibility [cf. Eq. (2.43)], and chooses a control $\tilde{u}_k \in U_k(\tilde{y}_k)$ according to the minimization

$$\tilde{u}_k \in \arg \min_{u_k \in U_k(\tilde{y}_k)} G\big(T_k(\tilde{y}_k, u_k)\big), \tag{2.47}$$

[cf. Eq. (2.45)], where

$$T_k(\tilde{y}_k, u_k) = \Big(\tilde{y}_k, u_k, R\big(\tilde{y}_k, u_k, f_k(\tilde{x}_k, u_k)\big)\Big);$$

[cf. Eq. (2.42)]. Finally, the algorithm sets

$$\tilde{x}_{k+1} = f_k(\tilde{x}_k, \tilde{u}_k), \qquad \tilde{y}_{k+1} = (\tilde{y}_k, \tilde{u}_k, \tilde{x}_{k+1}),$$

[cf. Eq. (2.44)], thus obtaining the partial trajectory $\tilde{y}_{k+1}$ to start the next stage.

It can be seen that our constrained rollout algorithm is not much more complicated or computationally demanding than its unconstrained version where the constraint $T \in C$ is not present (as long as checking feasibility of a complete trajectory T is not computationally demanding). Note, however, that our algorithm makes essential use of the deterministic character of the problem, and does not admit a straightforward extension to stochastic problems, since checking feasibility of a complete trajectory is typically difficult in the context of these problems.

The rollout algorithm just described is illustrated in Fig. 2.5.3 for our earlier traveling salesman Example 2.5.1. Here we want to find a minimum travel cost tour that additionally satisfies a safety constraint, namely that the "safety cost" of the tour should be less than a certain threshold. This cost may be given by a table as in Fig. 2.5.3, or it may be generated on line with an algorithm for any given tour. Note that the minimum cost tour, ABDCA, in this example does not satisfy the safety constraint. Moreover, the tour ABCDA obtained by the rollout algorithm has barely smaller cost than the tour ACDBA generated by the base heuristic starting from A. In fact if the travel cost D→A were larger, say 25, the tour produced by constrained rollout would be more costly than the one produced by the base heuristic starting from A. This points to the need for a constrained

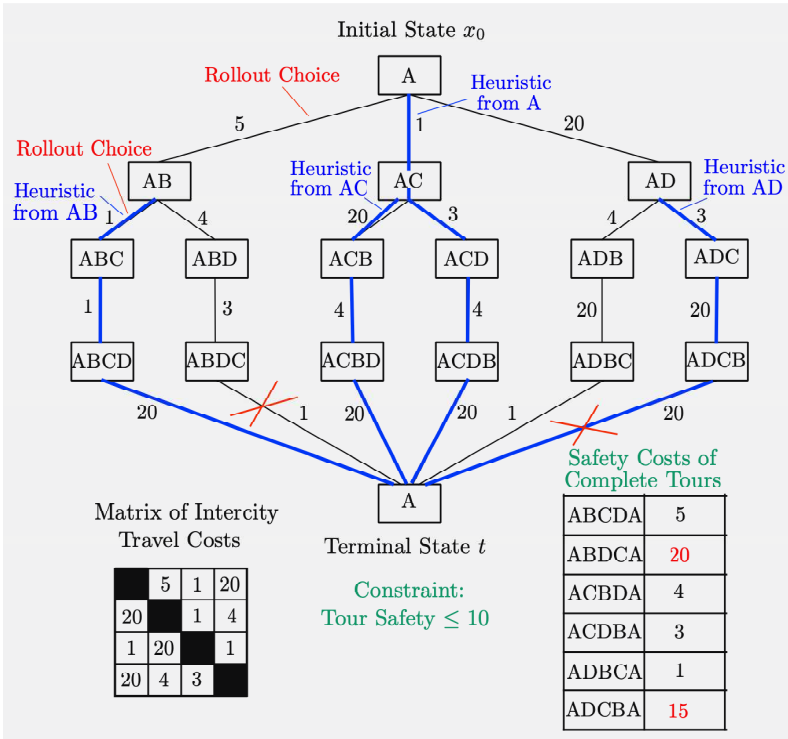

Figure 2.5.3 The constrained traveling salesman problem; cf. Example 2.5.1, and its rollout solution using the base heuristic shown, which completes a partial tour as follows:

At A it yields ACDBA.
At AB it yields ABCDA.
At AC it yields ACBDA.
At AD it yields ADCBA.

This base heuristic is not assumed to have any special structure. It is just capable of completing every partial tour without regard to any additional considerations. Thus for example the heuristic generates at A the complete tour ACDBA, and it switches to the tour ACBDA once the salesman moves to AC.

At city A, the rollout algorithm:

(a) Considers the partial tours AB, AC, and AD.

(b) Uses the base heuristic to obtain the corresponding complete tours ABCDA, ACBDA, and ADCBA.

(c) Discards ADCBA as being infeasible.

(d) Compares the other two tours, ABCDA and ACBDA, finds ABCDA to have smaller cost, and selects the partial tour AB.

(e) At AB, it considers the partial tours ABC and ABD.

(f) It uses the base heuristic to obtain the corresponding complete tours ABCDA and ABDCA, and discards ABDCA as being infeasible.

(g) It finally selects the complete tour ABCDA.

version of the notion of sequential improvement and for a fortified variant of the algorithm, which we discuss next.

Sequential Consistency, Sequential Improvement, and the Cost Improvement Property

We will now introduce sequential consistency and sequential improvement conditions guaranteeing that the control set $U_k(\tilde{y}_k)$ in the minimization (2.47) is nonempty, and that the costs of the complete trajectories

$$T_k(\tilde{y}_k, \tilde{u}_k)$$

are improving with each k in the sense that

$$G\big(T_{k+1}(\tilde{y}_{k+1}, \tilde{u}_{k+1})\big) \leq G\big(T_k(\tilde{y}_k, \tilde{u}_k)\big), \qquad k = 0, 1, \ldots, N-1,$$

while at the first step of the algorithm we have

$$G\big(T_0(\tilde{y}_0, \tilde{u}_0)\big) \leq G\big(R(\tilde{y}_0)\big).$$

It will then follow that the cost improvement property

$$G(\tilde{y}_N) \leq G\big(R(\tilde{y}_0)\big)$$

holds.

Definition 2.5.1: We say that the base heuristic is *sequentially consistent* if whenever it generates a partial trajectory

$$(x_k, u_k, x_{k+1}, u_{k+1}, \ldots, u_{N-1}, x_N),$$

starting from a partial trajectory y_k, it also generates the partial trajectory

$$(x_{k+1}, u_{k+1}, x_{k+2}, u_{k+2}, \ldots, u_{N-1}, x_N),$$

starting from the partial trajectory $y_{k+1} = \big(y_k, u_k, x_{k+1}\big)$.

As we have noted in the context of unconstrained rollout, greedy heuristics tend to be sequentially consistent. Also any policy [a sequence of feedback control functions $\mu_k(y_k)$, $k = 0, 1, \ldots, N-1$] for the DP problem of minimizing the terminal cost $G(y_N)$ subject to the system equation

$$y_{k+1} = \big(y_k, u_k, f_k(x_k, u_k)\big)$$

and the feasibility constraint $y_N \in C$ can be seen to be sequentially consistent. For an example where sequential consistency is violated, consider the base heuristic of the traveling salesman Example 2.5.1. From Fig. 2.5.3, it can be seen that the base heuristic at A generates ACDBA, but from AC it generates ACBDA, thus violating sequential consistency.

For a given partial trajectory y_k, let us denote by $y_k \cup R(y_k)$ the complete trajectory obtained by joining y_k with the partial trajectory generated by the base heuristic starting from y_k. Thus if

$$y_k = (x_0, u_0, \ldots, u_{k-1}, x_k)$$

and

$$R(y_k) = (x_k, u_k, \ldots, u_{N-1}, x_N),$$

we have

$$y_k \cup R(y_k) = (x_0, u_0, \ldots, u_{k-1}, x_k, u_k, \ldots, u_{N-1}, x_N).$$

Definition 2.5.2: We say that the base heuristic is *sequentially improving* if for every $k = 0, 1, \ldots, N-1$ and partial trajectory y_k for which $y_k \cup R(y_k) \in C$, the set $\hat{U}_k(y_k)$ is nonempty, and we have

$$G\big(y_k \cup R(y_k)\big) \geq \min_{u_k \in \hat{U}_k(y_k)} G\big(T_k(y_k, u_k)\big). \tag{2.48}$$

Note that for a base heuristic that is not sequentially consistent, the condition $y_k \cup R(y_k) \in C$ does not imply that the set $\hat{U}_k(y_k)$ is nonempty. The reason is that starting from the next state

$$y_{k+1} = \big(y_k, u_k, f_k(x_k, u_k)\big),$$

the base heuristic may generate a different trajectory than from y_k, even if it applies u_k at y_k. Thus we need to include nonemptiness of $\hat{U}_k(y_k)$ as a requirement in the preceding definition of sequential improvement (in the fortified version of the algorithm to be discussed shortly, this requirement will be removed).

On the other hand, if the base heuristic is sequentially consistent, it is also sequentially improving. The reason is that for a sequentially consistent heuristic, $y_k \cup R(y_k)$ is equal to one of the trajectories contained in the set

$$\big\{T_k(y_k, u_k) \mid u_k \in \hat{U}_k(y_k)\big\}.$$

Our main result is contained in the following proposition.

Proposition 2.5.1: (Cost Improvement for Constrained Rollout) Assume that the base heuristic is sequentially improving and generates a feasible complete trajectory starting from the initial state $\tilde{y}_0 = \tilde{x}_0$, i.e., $R(\tilde{y}_0) \in C$. Then for each k, the set $U_k(\tilde{y}_k)$ is nonempty, and we have

$$\begin{aligned} G\big(R(\tilde{y}_0)\big) &\geq G\big(T_0(\tilde{y}_0, \tilde{u}_0)\big) \\ &\geq G\big(T_1(\tilde{y}_1, \tilde{u}_1)\big) \\ &\geq \cdots \\ &\geq G\big(T_{N-1}(\tilde{y}_{N-1}, \tilde{u}_{N-1})\big) \\ &= G(\tilde{y}_N), \end{aligned}$$

where

$$T_k(\tilde{y}_k, \tilde{u}_k) = \big(\tilde{y}_k, \tilde{u}_k, R(\tilde{y}_{k+1})\big);$$

cf. Eq. (2.42). In particular, the final trajectory $\tilde{y}_N$ generated by the constrained rollout algorithm is feasible and has no larger cost than the trajectory $R(\tilde{y}_0)$ generated by the base heuristic starting from the initial state.

Proof: Consider $R(\tilde{y}_0)$, the complete trajectory generated by the base heuristic starting from $\tilde{y}_0$. Since $\tilde{y}_0 \cup R(\tilde{y}_0) = R(\tilde{y}_0) \in C$ by assumption, it follows from the sequential improvement definition, that the set $U_0(\tilde{y}_0)$ is nonempty and we have

$$G\big(R(\tilde{y}_0)\big) \geq G\big(T_0(\tilde{y}_0, \tilde{u}_0)\big),$$

[cf. Eq. (2.48)], while $T_0(\tilde{y}_0, \tilde{u}_0) \in C$.

The preceding argument can be repeated for the next stage, by replacing $\tilde{y}_0$ with $\tilde{y}_1$, and $R(\tilde{y}_0)$ with $T_0(\tilde{y}_0, \tilde{u}_0)$. Since $\tilde{y}_1 \cup R(\tilde{y}_1) = T_0(\tilde{y}_0, \tilde{u}_0) \in C$, from the sequential improvement definition, the set $U_1(\tilde{y}_1)$ is nonempty and we have

$$G\big(T_0(\tilde{y}_0, \tilde{u}_0)\big) = G\big(\tilde{y}_1 \cup R(\tilde{y}_1)\big) \geq G\big(T_1(\tilde{y}_1, \tilde{u}_1)\big),$$

[cf. Eq. (2.48)], while $T_1(\tilde{y}_1, \tilde{u}_1) \in C$. Similarly, the argument can be successively repeated for every k, to verify that $U_k(\tilde{y}_k)$ is nonempty and that $G\big(T_k(\tilde{y}_k, \tilde{u}_k)\big) \geq G\big(T_{k+1}(\tilde{y}_{k+1}, \tilde{u}_{k+1})\big)$ for all k. **Q.E.D.**

Proposition 2.5.1 establishes the fundamental cost improvement property for constrained rollout under the sequential improvement condition. On the other hand we may construct examples where the sequential improvement condition (2.48) is violated and the cost of the solution produced by rollout is larger than the cost of the solution produced by the

base heuristic starting from the initial state (cf. the unconstrained rollout Example 2.3.3).

In the case of the traveling salesman Example 2.5.1, it can be verified that the base heuristic specified in Fig. 2.5.3 is sequentially improving. However, if the travel cost D→A were larger, say 25, then it can be verified that the definition of sequential improvement would be violated at A, and the tour produced by constrained rollout would be more costly than the one produced by the base heuristic starting from A.

The Fortified Rollout Algorithm and Other Variations

We will now discuss some variations and extensions of the constrained rollout algorithm. Let us first consider the case where the sequential improvement assumption is not satisfied. Then it may happen that given the current partial trajectory $\tilde{y}_k$, the set of controls $U_k(\tilde{y}_k)$ that corresponds to feasible trajectories $T_k(\tilde{y}_k, u_k)$ [cf. Eq. (2.43)] is empty, in which case the rollout algorithm cannot extend the partial trajectory $\tilde{y}_k$ further. To bypass this difficulty, we introduce a *fortified constrained rollout algorithm*, patterned after the fortified algorithm given earlier. For validity of this algorithm, *we require that the base heuristic generates a feasible complete trajectory $R(\tilde{y}_0)$ starting from the initial state $\tilde{y}_0$.*

The fortified constrained rollout algorithm, in addition to the current partial trajectory

$$\tilde{y}_k = (\tilde{x}_0, \tilde{u}_0, \tilde{x}_1, \ldots, \tilde{u}_{k-1}, \tilde{x}_k),$$

maintains a complete trajectory $\hat{T}_k$, called *tentative best trajectory*, which is feasible (i.e., $\hat{T}_k \in C$) and agrees with $\tilde{y}_k$ up to state $\tilde{x}_k$, i.e., $\hat{T}_k$ has the form

$$\hat{T}_k = (\tilde{x}_0, \tilde{u}_0, \tilde{x}_1, \ldots, \tilde{u}_{k-1}, \tilde{x}_k, \overline{u}_k, \overline{x}_{k+1}, \ldots, \overline{u}_{N-1}, \overline{x}_N), \tag{2.49}$$

for some $\overline{u}_k, \overline{x}_{k+1}, \ldots, \overline{u}_{N-1}, \overline{x}_N$ such that

$$\overline{x}_{k+1} = f_k(\tilde{x}_k, \overline{u}_k), \qquad \overline{x}_{t+1} = f_t(\overline{x}_t, \overline{u}_t), \quad t = k+1, \ldots, N-1.$$

Initially, $\hat{T}_0$ is the complete trajectory $R(\tilde{y}_0)$, generated by the base heuristic starting from $\tilde{y}_0$, which is assumed to be feasible. At stage k, the algorithm forms the subset $\hat{U}_k(\tilde{y}_k)$ of controls $u_k \in U_k(\tilde{y}_k)$ such that the corresponding $T_k(\tilde{y}_k, u_k)$ is not only feasible, but also has cost that is no larger than the one of the current tentative best trajectory:

$$\hat{U}_k(\tilde{y}_k) = \Big\{ u_k \in U_k(\tilde{y}_k) \mid G\big(T_k(\tilde{y}_k, u_k)\big) \leq G(\hat{T}_k) \Big\}.$$

There are two cases to consider at state k:

(1) *The set $\hat{U}_k(\tilde{y}_k)$ is nonempty.* Then the algorithm forms the partial trajectory $\tilde{y}_{k+1} = (\tilde{y}_k, \tilde{u}_k, \tilde{x}_{k+1})$, where

$$\tilde{u}_k \in \arg \min_{u_k \in \hat{U}_k(\tilde{y}_k)} G\big(T_k(\tilde{y}_k, u_k)\big), \qquad \tilde{x}_{k+1} = f_k(\tilde{x}_k, \tilde{u}_k),$$

and sets $T_k(\tilde{y}_k, \tilde{u}_k)$ as the new tentative best trajectory, i.e.,

$$\hat{T}_{k+1} = T_k(\tilde{y}_k, \tilde{u}_k).$$

(2) *The set $\hat{U}_k(\tilde{y}_k)$ is empty.* Then, the algorithm forms the partial trajectory $\tilde{y}_{k+1} = \big(\tilde{y}_k, \tilde{u}_k, \tilde{x}_{k+1}\big)$, where

$$\tilde{u}_k = \overline{u}_k, \qquad \tilde{x}_{k+1} = \overline{x}_{k+1},$$

and $\overline{u}_k, \overline{x}_{k+1}$ are the control and state subsequent to $\tilde{x}_k$ in the current tentative best trajectory $\hat{T}_k$ [cf. Eq. (2.49)], and leaves $\hat{T}_k$ unchanged, i.e.,

$$\hat{T}_{k+1} = \hat{T}_k.$$

It can be seen that the fortified constrained rollout algorithm will follow the initial complete trajectory $\hat{T}_0$, the one generated by the base heuristic starting from $\tilde{y}_0$, up to a stage k where it will discover a new feasible complete trajectory with smaller cost to replace $\hat{T}_0$ as the tentative best trajectory. Similarly, the new tentative best trajectory $\hat{T}_k$ may be subsequently replaced by another feasible trajectory with smaller cost, etc.

Note that if the base heuristic is sequentially improving, and the fortified rollout algorithm will generate the same complete trajectory as the (nonfortified) rollout algorithm given earlier, with the tentative best trajectory $\hat{T}_{k+1}$ being equal to the complete trajectory $T_k(\tilde{y}_k, \tilde{u}_k)$ for all k. The reason is that if the base heuristic is sequentially improving, the controls $\tilde{u}_k$ generated by the nonfortified algorithm belong to the set $\hat{U}_k(\tilde{y}_k)$ [by Prop. 2.5.1, case (1) above will hold].

However, it can be verified that even when the base heuristic is not sequentially improving, the fortified rollout algorithm will generate a complete trajectory that is feasible and has cost that is no worse than the cost of the complete trajectory generated by the base heuristic starting from $\tilde{y}_0$. This is because each tentative best trajectory has a cost that is no worse than the one of its predecessor, and the initial tentative best trajectory is just the trajectory generated by the base heuristic starting from the initial condition $\tilde{y}_0$.

Tree-Based Constrained Rollout Algorithms

It is possible to improve the performance of the rollout algorithm at the expense of maintaining more than one partial trajectory. In particular,

instead of the partial trajectory $\tilde{y}_k$ of Eq. (2.46), we can maintain a *tree* of partial trajectories that is rooted at $\tilde{y}_0$. These trajectories need not have equal length, i.e., they need not involve the same number of stages. At each step of the algorithm, we select a single partial trajectory from this tree, and execute the rollout algorithm's step as if this partial trajectory were the only one. Let this partial trajectory have k stages and denote it by $\tilde{y}_k$. Then we extend $\tilde{y}_k$ similar to our earlier rollout algorithm, with possibly multiple feasible trajectories. There is also a fortified version of this algorithm where a tentative best trajectory is maintained, which is the minimum cost complete trajectory generated thus far.

The aim of the tree-based algorithm is to obtain improved performance, essentially because it can go back and extend partial trajectories that were generated and temporarily abandoned at previous stages. The net result is a more flexible algorithm that is capable of examining more alternative trajectories. Note also that there is considerable freedom to select the number of partial trajectories maintained in the tree.

We finally mention a drawback of the tree-based algorithm: it is suitable for off-line computation, but it cannot be applied in an on-line context, where the rollout control selection is made after the current state becomes known as the system evolves in real-time .

2.5.1 Constrained Rollout for Discrete Optimization and Integer Programming

As noted in Section 2.1, general discrete optimization problems may be formulated as DP problems, which in turn can be addressed with rollout. The following is an example of a classical problem that involves both discrete and continuous variables. It can also be viewed as an instance of a 0-1 integer programming problem, and in fact this is the way it is usually addressed in the literature; see e.g., the book [DrH01]. The author's rollout book [Ber20a] contains additional examples.

Example 2.5.2 (Facility Location)

We are given a candidate set of N locations, and we want to place in some of these locations a "facility" that will serve the needs of a total of M "clients." Each client $i = 1, \ldots, M$ has a demand d_i for services that may be satisfied at a location $k = 0, \ldots, N-1$ at a cost a_{ik} per unit. If a facility is placed at location k, it has capacity to serve demand up to a known level c_k.

We introduce a 0-1 integer variable u_k to indicate with $u_k = 1$ that a facility is placed at location k at a cost b_k and with $u_k = 0$ that a facility is not placed at location k. Thus if y_{ik} denotes the amount of demand of client i to be served at facility k, the constraints are

$$\sum_{k=0}^{N-1} y_{ik} = d_i, \qquad i = 1, \ldots, M, \tag{2.50}$$

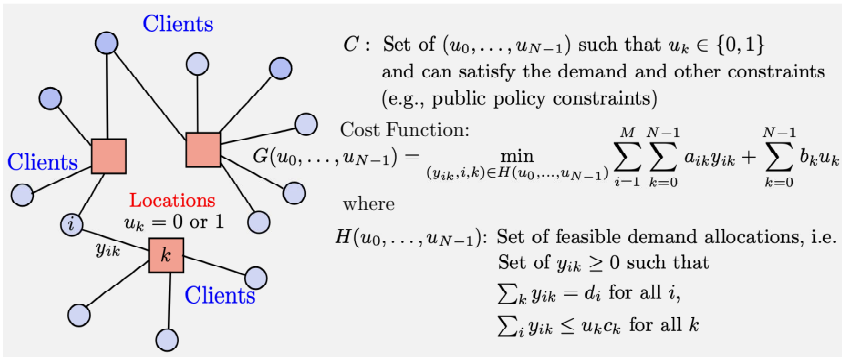

Figure 2.5.4 Schematic illustration of the facility location problem; cf. Example 2.5.2. Clients are matched to facilities, and the locations of the facilities are subject to optimization.

$$\sum_{i=1}^{M} y_{ik} \leq c_k u_k, \qquad k = 0, \ldots, N-1, \tag{2.51}$$

together with

$$y_{ik} \geq 0, \quad u_k \in \{0, 1\}, \qquad i = 1, \ldots, M, \ k = 0, \ldots, N-1. \tag{2.52}$$

We wish to minimize the cost

$$\sum_{i=1}^{M} \sum_{k=0}^{N-1} a_{ik} y_{ik} + \sum_{k=0}^{N-1} b_k u_k \tag{2.53}$$

subject to the preceding constraints; see Fig. 2.5.4. The essence of the problem is to place enough facilities at favorable locations to satisfy the clients' demand at minimum cost. This can be a very difficult mixed integer programming problem.

On the other hand, when all the variables u_k are fixed at some 0 or 1 values, the problem belongs to the class of *linear transportation* problems (see e.g., [Ber98]), and can be solved by fast polynomial algorithms. Thus the essential difficulty of the problem is how to select the integer variables u_k, $k = 0, \ldots, N-1$. This can be viewed as a discrete optimization problem of the type discussed in Section 1.6.3 (cf. Fig. 1.6.2). In terms of the notation of this figure, the control components are $u_0, \ldots, u_{N-1}$, where u_k can take the values 0 or 1.

To address the problem suboptimally by rollout, we must define a base heuristic at a "state" $(u_0, \ldots, u_k)$, where $u_j = 1$ or $u_j = 0$ specifies that a facility is or is not placed at location j, respectively. A suitable base heuristic at that state is to place a facility at all of the remaining locations (i.e., $u_j = 1$ for $j = k+1, \ldots, N-1$), and its cost is obtained by solving the corresponding linear transportation problem of minimizing the cost (2.53) subject to the constraints (2.50)-(2.52), with the variables u_j, $j = 0, \ldots, k$, fixed at the previously chosen values, and the variables u_j, $j = k+1, \ldots, N$, fixed at 1.

To illustrate, at the initial state where no placement decision has been made, we set $u_0 = 1$ (a facility is placed at location 0) or $u_0 = 0$ (a facility is not placed at location 0), we solve the two corresponding transportation problems, and we fix u_0, depending on which of the two resulting costs is smallest. Having fixed the status of location 0, we repeat with location 1: set the variable u_1 to 1 and to 0, solve the corresponding two transportation problems, and fix u_1, depending on which of the two resulting costs is smallest, etc.

It is easily seen that if the initial base heuristic choice (placing a facility at every candidate location) is feasible, i.e.,

$$\sum_{i=1}^{M} d_i \leq \sum_{k=0}^{N-1} c_k,$$

the rollout algorithm will yield a feasible solution with cost that is no larger than the cost corresponding to the initial application of the base heuristic. In fact it can be verified that the base heuristic here is sequentially consistent, so it is not necessary to use the fortified version of the algorithm. Regarding computational costs, the number of transportation problems to be solved is at first count $2N$, but it can be reduced to $N + 1$ by exploiting the fact that one of the two transportation problems at each stage after the first has been solved at an earlier stage.

It is worth noting, for readers that are familiar with the integer programming method of branch-and-bound, that the graph of Fig. 2.1.4 corresponds to the branch-and-bound tree for the problem, so the rollout algorithm amounts to a quick (and imperfect) method to traverse the branch-and-bound tree. This observation may be useful if we wish to use integer programming techniques to add improvements to the rollout algorithm.

We finally note that the rollout algorithm requires the solution of many linear transportation problems, which are defined by fairly similar data. It is thus important to use an algorithm that is capable of using effectively the final solution of one transportation problem as a starting point for the solution of the next. The auction algorithm for transportation problems (Bertsekas and Castañon [BeC89]) is particularly well-suited for this purpose.

Example 2.5.3 (Constrained Shortest Paths and Directed Spanning Trees)

Let us consider a spanning tree-type problem involving a directed graph with nodes $0, 1, \ldots, N$. At each node $k \in \{0, \ldots, N-1\}$ there is a set of outgoing arcs $u_k \in U_k$. Node N is special: it is viewed as a "root" node and has no outgoing arc. We are interested in collections of arcs involving a single outgoing arc per node,

$$u = (u_0, \ldots, u_{N-1})$$

with $u_k \in U_k$, $k = 0, \ldots, N-1$. We require that these arcs do not form a cycle, so that u specifies a directed spanning tree that is rooted at node

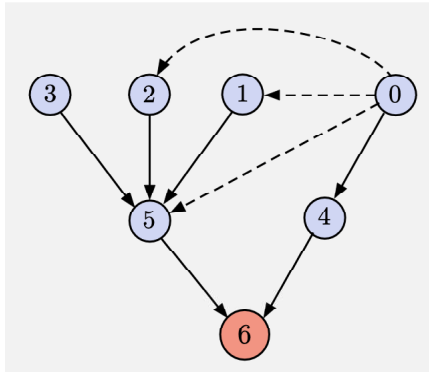

Figure 2.5.5 Schematic illustration of a constrained shortest path problem with root node $N = 6$. Given the current feasible spanning tree solution (indicated with solid line arcs), the rollout algorithm, considers a node k (in the figure $k = 0$) and the spanning tree arcs $\{u_i \mid i \neq k\}$ that are outgoing from the nodes $i \neq k$. It then considers the spanning trees that correspond to the outgoing arcs u_k from k that do not close a cycle with the set $\{u_i \mid i \neq k\}$ and are feasible [in the figure, these are the arcs indicated with broken lines, plus the arc (0,4)], and selects the arc that forms a spanning tree solution of minimum cost.

N. Note that for every node k, such a spanning tree specifies a unique path that starts at k, lies on the spanning tree, and ends at node N. We wish to find u that minimizes a given cost function $G(u)$ subject to certain additional constraints, which we do not specify further. The set of all constraints on u (including the constraint that the arcs form a directed spanning tree) is denoted abstractly as $u \in U$, so the problem comes within our constrained optimization framework of this section.

Note that this problem contains as a special case the classical shortest path problem, where we have a length for every arc and the objective is to find a tree of shortest paths to node N from all the nodes $0, \ldots, N-1$. Here U is just the constraint that the set of arcs $u = (u_0, \ldots, u_{N-1})$ form a directed spanning tree that is rooted at node N, and $G(u)$ is the sum of the lengths of all the paths specified by u, summed over all the start nodes $k = 0, \ldots, N-1$. Other shortest path-type problems, involving constraints, are included as special cases. For example, there may be a constraint that all the paths to N that are specified by the spanning tree corresponding to u contain a number of arcs that does not exceed a given upper bound.

Suppose that we have an initial solution/directed spanning tree

$$\bar{u} = (\bar{u}_0, \ldots, \bar{u}_{N-1}),$$

which is feasible (note here that finding such an initial solution may be a challenge). Let us apply the constrained rollout algorithm with a base heuristic that operates as follows: given a partial trajectory

$$y_k = (u_0, \ldots, u_{k-1}),$$

i.e., a sequence of k arcs, each outgoing from one of the nodes $0, \ldots, k-1$, it generates the complete trajectory/directed spanning tree

$$(u_0, \ldots, u_{k-1}, \bar{u}_k, \ldots, \bar{u}_{N-1}).$$

Thus the rollout algorithm, given a partial trajectory

$$\tilde{y}_k = (\tilde{u}_0, \ldots, \tilde{u}_{k-1}),$$

considers the set $\hat{U}_k(\tilde{y}_k)$ of all outgoing arcs u_k from node k, such that the complete trajectory

$$(\tilde{y}_k, u_k, \bar{u}_{k+1}, \ldots, \bar{u}_{N-1})$$

is feasible. It then selects the arc $u_k \in \hat{U}_k(\tilde{y}_k)$ that minimizes the cost

$$G(\tilde{y}_k, u_k, \bar{u}_{k+1}, \ldots, \bar{u}_{N-1});$$

see Fig. 2.5.5. It can be seen by induction, starting from $\bar{u}$, that the set of arcs $\hat{U}_k(\tilde{y}_k)$ is nonempty, and that the algorithm generates a sequence of feasible solutions/spanning trees, each with cost no worse than the preceding one.

Note that throughout the rollout process, a rooted spanning tree is maintained, and at each stage k, a single arc $\bar{u}_k$ that is outgoing from node k is replaced by the outgoing arc $\tilde{u}_k$. Thus two successive rooted spanning trees generated by the algorithm, differ by at most a single arc.

An interesting aspect of this rollout algorithm is that it can be applied multiple times with the final solution of one rollout application used to specify the base heuristic of the next rollout application. Moreover, a different order of nodes may be used in each rollout application. This can be viewed as a form of policy iteration, of the type that we have discussed. The algorithm will eventually terminate, in the sense that it can make no further progress. More irregular/heuristic orders of node selections are also possible; for example some nodes may be selected multiple times before others will be selected for the first time. However, there is no guarantee that the final solution thus obtained will be optimal.

2.6 SMALL STAGE COSTS AND LONG HORIZON - CONTINUOUS-TIME ROLLOUT

Let us consider the deterministic one-step approximation in value space scheme

$$\tilde{\mu}_k(x_k) \in \arg \min_{u_k \in U_k(x_k)} \Big[g_k(x_k, u_k) + \tilde{J}_{k+1}\big(f_k(x_k, u_k)\big) \Big]. \tag{2.54}$$

In the context of rollout, $\tilde{J}_{k+1}\big(f_k(x_k, u_k)\big)$ is either the cost of the trajectory generated by the base heuristic starting from the next state $f_k(x_k, u_k)$, or some approximation that may involve truncation and terminal cost function approximation, as in the truncated rollout scheme of Section 2.3.6.

There is a special difficulty in this context, which is often encountered in practice. It arises when the cost per stage $g_k(x_k, u_k)$ is either 0 or is small relative to the cost-to-go approximation $\tilde{J}_{k+1}\big(f_k(x_k, u_k)\big)$. Then there is a potential pitfall to contend with: *the cost approximation errors that are inherent in the term* $\tilde{J}_{k+1}\big(f_k(x_k, u_k)\big)$ *may overwhelm the first stage cost term* $g_k(x_k, u_k)$, with unpredictable consequences for the quality of the one-step-lookahead policy $\tilde{\pi} = \{\tilde{\mu}_0, \ldots, \tilde{\mu}_{N-1}\}$.

The most straightforward way to address this issue is to use longer lookahead; this is typically what is done in the context of MPC (cf. Section 1.6.9). The difficulty here is that long lookahead minimization may require extensive on-line computation. In this case, creative application of the lookahead tree pruning and incremental rollout ideas, discussed in Section 2.4.2, may be helpful.

We will next discuss the difficulty with small stage costs for an alternative context, which arises from discretization of a continuous-time optimal control problem.

Continuous-Time Optimal Control and Approximation in Value Space

Consider a problem that involves a vector differential equation of the form

$$\dot{x}(t) = h\big(x(t), u(t), t\big), \qquad 0 \le t \le T, \tag{2.55}$$

where $x(t) \in \Re^n$ is the state vector at time t, $\dot{x}(t) \in \Re^n$ is the vector of first order time derivatives of the state at time t, $u(t) \in U \subset \Re^m$ is the control vector at time t, where U is the control constraint set, and T is a given terminal time. Starting from a given initial state $x(0)$, we want to find a feasible control trajectory $\big\{u(t) \mid t \in [0, T]\big\}$, which together with its corresponding state trajectory $\big\{x(t) \mid t \in [0, T]\big\}$, minimizes a cost function of the form

$$G\big(x(T)\big) + \int_0^T g\big(x(t), u(t), t\big)\,dt, \tag{2.56}$$

where g represents cost per unit time, and G is a terminal cost function. This is a classical problem with a long history.

Let us consider a simple conversion of the preceding continuous-time problem to a discrete-time problem, while treading lightly over some of the associated mathematical fine points. We introduce a small discretization increment $\delta > 0$, such that $T = \delta N$ where N is a large integer, and we replace the differential equation (2.55) by

$$x_{k+1} = x_k + \delta \cdot h_k(x_k, u_k), \qquad k = 0, \ldots, N-1.$$

Here the function h_k is given by

$$h_k(x_k, u_k) = h\big(x(k\delta), u(k\delta), k\delta\big),$$